

Automated Code Review Using a Multi-LLMs Framework

Po-Yun Cheng^{1*} Kuan-Yi Lee¹ Chen-Wei Zhang¹ Jonathan Lee¹

¹ Department of Computer Science and Information Engineering, National Taiwan University

*Email: b11902038@ntu.edu.tw

Abstract—AI-generated code now outpaces human reviewers’ ability to verify it, making automated review tools increasingly important. While most existing approaches rely on a single LLM, we propose LLMCR, a multi-LLM code review system that simulates four human reviewer reasoning steps: interpreting changes, planning checks, analyzing items, and summarizing. This separates context across stages, keeps reasoning transparent, and allows cheaper models to be used where appropriate. We evaluate LLMCR on 10 Spring AI pull requests against existing tools such as CodeRabbit and GitHub Copilot, using two similarity-based metrics: comment-alignment and interpretation-alignment. LLMCR achieves the highest comment-alignment F1 score at substantially lower cost than CodeRabbit and Copilot, though its interpretation-alignment score remains lower than Copilot’s, likely due to missing project context (e.g., issues, conventions, design rationale) in current datasets. We discuss this limitation and outline future directions for richer context construction.

Index Terms—Automated Code Review, Large Language Model, Multi-Agent System

I. INTRODUCTION

AI generates code at a faster pace than human reviewers can verify, so an automated system is important to increase reviewers’ efficiency and enhance review quality. Most related works focus on a single LLM combined with different prompting, retrieval-augmented generation, or tool-calling techniques.

We propose LLMCR, an intelligent multi-LLM code review system that focuses on validating Java pull requests and delivering comprehensive code reviews, helping reviewers evaluate code more efficiently. Figure 1 shows a generated review from LLMCR, where it produces a comprehensive review with the necessary information for knowledge transfer within teams.

II. METHODOLOGY

A. Stage-based Code Review System

We investigate the reasoning process of a human reviewer and summarize it into four reasoning steps: 1) Interpreting code changes, 2) Planning what to check, 3) Analyzing individual check items and 4) Summary the results into a review.

We then design our multi-LLM framework, where each stage maps to a specific human reasoning step, as shown in figure 2. This design allows us to:

- Separate the information needed for different stages, avoiding putting all context into a single iteration.
- Keep the process transparent, ensuring the model follows the same reasoning process as a human.
- Allow different stages to use specialized models.



Fig. 1. Example output review of LLMCR. It contains the following sections: *Motivation*, explaining why the change was made; *Suggestions*, offering concrete recommendations for improvement; *Good Points* and *Bad Points*, summarizing what can be learned from the change for knowledge transfer; *Implementation Details*, showing what was changed in each file; and *Issues*, listing every issue identified in the code change.

At each stage, we use retrieval-augmented generation (RAG) to provide relevant project context and knowledge to the model, enabling more informed and context-aware code review. For InterpretationLM, PlanningLM, and SummaryLM, we use Gemini-3.1-flash-lite. For ComputationLM and RetrievalLM, we use Nvidia-Nemotron-3-4B to reduce cost.

B. ETL Pipeline and Datasets

For any project to be used, we first index its project context using an Extract, Transform, Load (ETL) pipeline as shown in

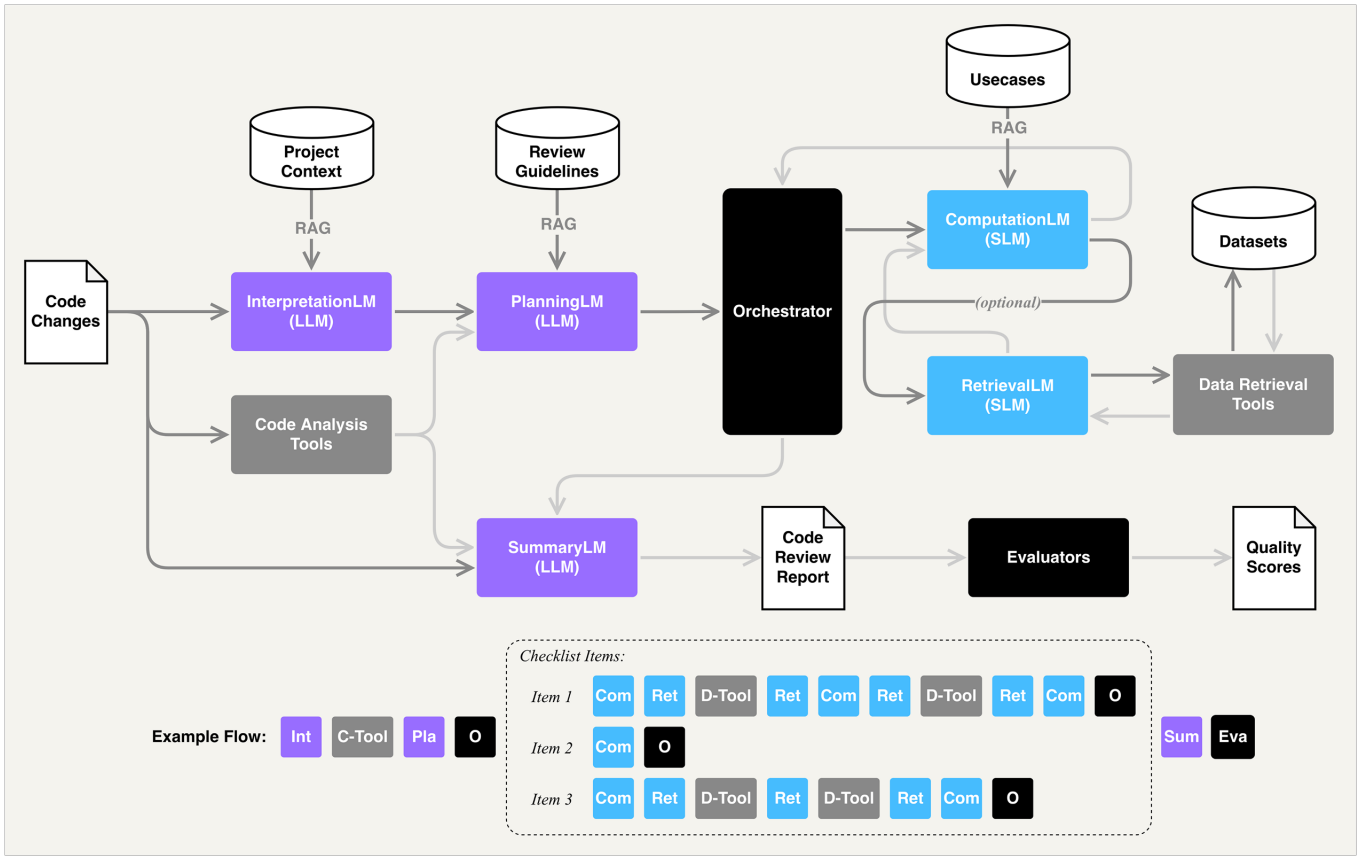


Fig. 2. Overview of our stage-based code review system.

figure 3. The ETL pipeline is responsible for extracting paragraphs from documents, transforming them into semantically meaningful chunks, and loading these chunks into a vector database. We currently use Spring AI as our test project, using its full source code repository and documentation as the project context.

We use MariaDB as our main database and FAISS (Facebook AI Similarity Search) for vector search. We apply different chunking techniques depending on file extension: for .java files, we treat each class as an independent context; for .md or .adoc files, we separate by section; for .pdf files, we separate by text size.

We also apply enrichment to Java classes by using an LLM (Gemini-2.5-flash-lite) to generate a summary of each class’s functionality, allowing Java classes to be found via semantic similarity. Since AI generates code at a faster pace than human reviewers can verify it, an automated system is important to increase reviewers’ efficiency and enhance review quality.

III. EVALUATION

We evaluate our system on 10 real pull requests from Spring AI’s GitHub repository. These pull requests are selected using the following criteria: 1) the pull request description is more than 50 words, and 2) the pull request has more than 10 human comments. We compare our system with three different

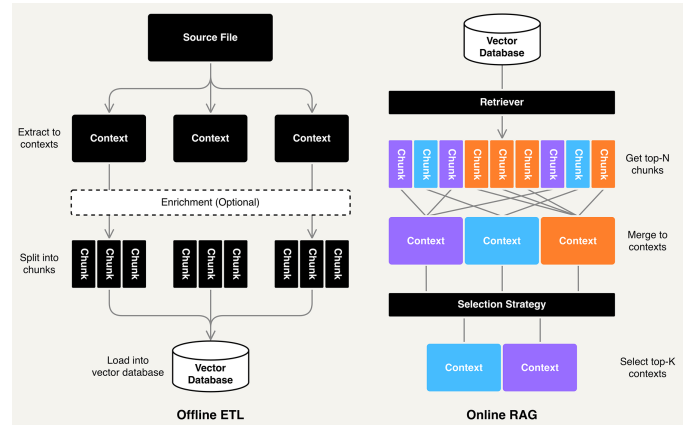


Fig. 3. Offline ETL and online RAG workflows.

groups: CodeRabbit and GitHub Copilot, which are widely-used pull request review tools, and SingleLLM, which is our system but with only the summary stage performed. Two evaluation metrics are used: *commentalignment* and *interpretationalignment*.

Comment-alignment is used to evaluate whether the generated review mentions things similar to human comments. We first use an LLM to rephrase human comments into clear,

atomic sentences as references. For the generated review, we separate it into candidate sentences and then use Sentence-BERT to determine whether each candidate-reference pair is similar. The recall, precision, and F1 score are calculated as below:

$$\text{Recall} = \frac{\text{number of similar pairs}}{\text{number of references}} \quad (1)$$

$$\text{Precision} = \frac{\text{number of similar pairs}}{\text{number of candidates}} \quad (2)$$

$$\text{F1} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (3)$$

Interpretation-alignment is used to evaluate whether the generated code-change interpretation aligns with the original pull request description. The calculation method is the same as *commentalignment*, but using the pull request description as the reference.

IV. RESULT AND ANALYSIS

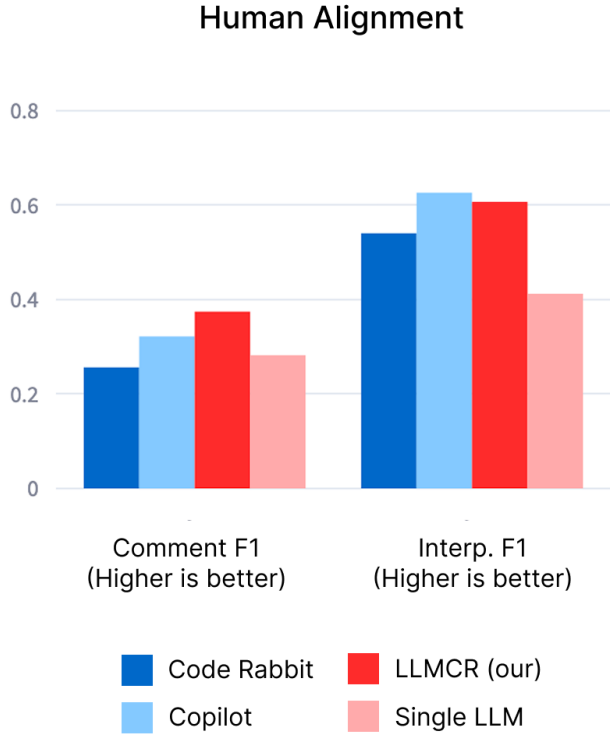


Fig. 4. Comment F1 and interpretation F1 for LLMCR (ours), Single LLM, CodeRabbit, and GitHub Copilot.

A. Human Alignment

The evaluation results are shown in Figure 4. For *comment-alignment*, LLMCR achieved the highest F1 score among all groups, suggesting that reviews generated by LLMCR are more aligned with human comments. For *interpretation-alignment*, LLMCR has a lower F1 score than Copilot and

a slightly higher score than CodeRabbit. This lower F1 score indicates that our system is not yet better at understanding code changes. We believe the primary reason is that the datasets used lack the necessary information from the start.

Currently, we only use source code and API documents in our datasets. However, for the review to be context-aware, the following contextual data is also required, which is difficult to obtain from existing datasets:

- Current issues or future plans of the project.
- Coding conventions.
- Project decision considerations.

Without this information, the LLM cannot correctly identify why a change was made and what problem it aims to solve, what constitutes the intended design for the project’s needs versus what constitutes an actual defect, or what aspects to prioritize when facing trade-offs (e.g., performance vs. consistency). Additionally, we currently chunk documentation only by paragraph, which may provide incomplete information to the LLM. The review guidelines provided to the LLM are also not curated and may contain noisy content.

B. System Efficiency and Cost

Table I shows the average review time and cost for the four systems on the 10 tested pull request reviews. GitHub Copilot gives the fastest review time (5 minutes per thousand lines of change) while having the highest cost. Single LLM is the most efficient and lowest-cost option, but its output quality (human alignment) is also the lowest. Overall, LLMCR demonstrates the most cost-effective performance, achieving similar output quality and review time to the two existing tools while incurring lower cost.

V. CONCLUSION

In this work, we introduce LLMCR, an automated code review system powered by a multiLLM framework. It achieves performance comparable to existing code review tools while using less powerful models and incurring lower inference costs. However, the designed system fails to effectively provide the project context required by the LLMs, due to the lack of suitable datasets.

For future work, we plan several directions to partially mitigate this data limitation: 1) collect GitHub issues to better understand current project concerns, 2) infer coding conventions from source code and existing documents, 3) infer development considerations from past review comments, and 4) manually rewrite and organize the review guidelines. In addition, we aim to construct a larger and more robust evaluation dataset to enable a more comprehensive and reliable assessment of code review quality.

REFERENCES

- [1] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, “Modern code review: a case study at google,” in Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, in ICSE-SEIP ’18. New York, NY, USA: Association for Computing Machinery, May 2018, pp. 181–190. doi: 10.1145/3183519.3183525.

TABLE I
REVIEW TIME, INPUT AND OUTPUT TOKENS, COST

Groups	Review Time (per thousand lines of change)	Input and Output Tokens (per review)	Cost (per review)
LLMCR	~10 minutes	(gemini-3.1-flash-lite) 61452 / 1568 (nemotron-3-4b) 1820242 / 4905	0.02 usd
Single LLM	<10s	(gemini-3.1-flash-lite) 14615 / 2069	0.007 usd
CodeRabbit	~10 minutes	x	0.25 usd
GitHub Copilot	~5 minutes	x	0.5 usd

- [2] M. N. Kapadnis, A. Naik, and C. Rose, “CRScore++: Reinforcement Learning with Verifiable Tool and AI Feedback for Code Review,” May 30, 2025, arXiv: arXiv:2506.00296. doi: 10.48550/arXiv.2506.00296.
- [3] A. Bosu, M. Greiler, and C. Bird, “Characteristics of Useful Code Reviews: An Empirical Study at Microsoft,” in 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, May 2015, pp. 146–156. doi: 10.1109/MSR.2015.21.
- [4] [1] P. Ersoy and M. Erşahin, “A Deep Dive into LLM-Powered Code Review Tools: A Comparative Analysis,” vol. 5, no. 2, 2024.
- [5] spring-projects/spring-ai. (Jun. 16, 2026). Java. Spring. Accessed: Jun. 16, 2026. [Online]. Available: <https://github.com/spring-projects/spring-ai>